

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks:
50			
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5

5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5
8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5
9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5

Code of Conduct:

I D. Gnana Deepthi (192411123) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

D. Gnanadeepthi.

ANSWERS

1. Pseudocode for Complete Query Processing Pipeline

ALGORITHM QueryProcessingPipeline(SQL_Query)

BEGIN

// Step 1: Receive SQL Query

INPUT SQL_Query

// Step 2: Lexical Analysis

Tokens ← Tokenize(SQL_Query)

// Step 3: Parsing

Parse_Tree ← Parse(Tokens)

IF Parse_Tree contains syntax error THEN

 DISPLAY "Syntax Error in SQL Query"

 STOP

END IF

// Step 4: Semantic Analysis

Valid ← CheckTables(Parse_Tree)

Valid ← CheckAttributes(Parse_Tree)

Valid ← CheckDataTypes(Parse_Tree)

Valid ← CheckUserPermissions(Parse_Tree)

IF Valid = FALSE THEN

 DISPLAY "Semantic or Authorization Error"

 STOP

END IF

// Step 5: Convert SQL into Logical Query Plan

Logical_Plan ← GenerateLogicalPlan(Parse_Tree)

// Step 6: Query Optimization

Optimized_Plan ← Logical_Plan

Optimized_Plan ← PushDownSelections(Optimized_Plan)

Optimized_Plan ← PushDownProjections(Optimized_Plan)

Optimized_Plan ← ReorderJoins(Optimized_Plan)

Optimized_Plan ← SelectBestAccessPaths(Optimized_Plan)

Optimized_Plan ← EstimateCostAndChooseBestPlan(Optimized_Plan)

```

// Step 7: Generate Physical Execution Plan
Physical_Plan ← GeneratePhysicalPlan(Optimized_Plan)

// Step 8: Execute Query
FOR each operator IN Physical_Plan DO
    Execute(operator)
END FOR

Result ← CollectOutput(Physical_Plan)

// Step 9: Format and Display Results
Formatted_Result ← Format(Result)
DISPLAY Formatted_Result

END

```

Flow of Query Processing

SQL Input → Lexical Analysis → Parsing → Semantic Checking → Logical Plan → Query Optimization → Physical Execution Plan → Execution → Result Output

2. Pseudocode for Cost-Based Query Optimization

ALGORITHM CostBasedJoinOptimization(Relations, JoinConditions)

```

BEGIN

    BestPlan ← NULL
    MinimumCost ← INFINITY

    // Generate all possible join orderings
    JoinOrders ← GenerateAllJoinOrders(Relations)

    FOR EACH Order IN JoinOrders DO

        TotalCost ← 0
        CurrentResult ← Order[0]

        FOR i ← 1 TO Length(Order) - 1 DO

            NextRelation ← Order[i]

```

```

// Estimate I/O cost for the current join
ScanCost ← EstimateScanCost(CurrentResult)
           + EstimateScanCost(NextRelation)

JoinCost ← EstimateJoinCost(
    CurrentResult,
    NextRelation,
    JoinConditions
)

// Calculate intermediate result size
ResultSize ← EstimateResultSize(
    CurrentResult,
    NextRelation,
    JoinConditions
)

// Add I/O costs
TotalCost ← TotalCost + ScanCost + JoinCost

// Store intermediate result
CurrentResult ← CreateIntermediateResult(ResultSize)

END FOR

// Compare with the best plan found so far
IF TotalCost < MinimumCost THEN
    MinimumCost ← TotalCost
    BestPlan ← Order
END IF

END FOR

DISPLAY "Best Join Order: ", BestPlan
DISPLAY "Estimated Minimum I/O Cost: ", MinimumCost

RETURN BestPlan

END

```

Supporting Functions

```

FUNCTION EstimateScanCost(Relation)
    RETURN NumberOfDiskBlocks(Relation)
END FUNCTION

```

```
FUNCTION EstimateJoinCost(Relation1, Relation2, JoinConditions)
```

```
    // Example: estimate Nested Loop Join I/O cost
```

```
    B1 ← NumberOfDiskBlocks(Relation1)
```

```
    B2 ← NumberOfDiskBlocks(Relation2)
```

```
    RETURN B1 + (B1 × B2)
```

```
END FUNCTION
```

```
FUNCTION EstimateResultSize(Relation1, Relation2, JoinConditions)
```

```
    // Estimate number of tuples in intermediate result
```

```
    T1 ← NumberOfTuples(Relation1)
```

```
    T2 ← NumberOfTuples(Relation2)
```

```
    Selectivity ← EstimateSelectivity(JoinConditions)
```

```
    RETURN T1 × T2 × Selectivity
```

```
END FUNCTION
```

Working

1. Generate all possible join orders.
2. Estimate the **scan I/O cost** and **join I/O cost** for each ordering.
3. Estimate the size of each intermediate result.
4. Add all I/O costs for a complete join plan.
5. Compare the total costs of all plans.
6. Select the join ordering with the **minimum estimated I/O cost**.

Example: For relations **A**, **B**, and **C**, the algorithm compares orders such as:

$(A \bowtie B) \bowtie C$

$(A \bowtie C) \bowtie B$

$(B \bowtie C) \bowtie A$

The plan with the lowest estimated I/O cost is selected for execution.

3. Nested Loop Join Algorithm and Complexity Analysis

Pseudocode

```
ALGORITHM NestedLoopJoin(R, S, JoinCondition)
```

```
BEGIN
```

```
    Result ← Empty
```

```

FOR EACH tuple r IN R DO

    FOR EACH tuple s IN S DO

        IF JoinCondition(r, s) = TRUE THEN
            Add (r, s) to Result
        END IF

    END FOR

END FOR

RETURN Result

END

Buffer Page Version
ALGORITHM BlockNestedLoopJoin(R, S, B)

BEGIN

    Result  $\leftarrow$  Empty

    // B = Number of available buffer pages
    BufferSize  $\leftarrow$  B - 2

    FOR EACH block of BufferSize pages from R DO

        Load block of R into memory

        FOR EACH page of S DO

            Load page of S into buffer

            FOR EACH tuple r in memory block DO
                FOR EACH tuple s in current page of S DO

                    IF JoinCondition(r, s) = TRUE THEN
                        Add (r, s) to Result
                    END IF

                END FOR
            END FOR

        END FOR
    END FOR

```

END FOR

END FOR

RETURN Result

END

Complexity Analysis

Let:

- M = Number of buffer pages of relation R
- N = Number of buffer pages of relation S
- $T(R)$ = Number of tuples in R
- $T(S)$ = Number of tuples in S
- B = Number of available buffer pages

Tuple Nested Loop Join

The time complexity is:

$O(T(R) \times T(S))$

The approximate I/O cost is:

$M + (T(R) \times N)$

This occurs because each tuple of the outer relation may require scanning all pages of the inner relation.

Block Nested Loop Join

The I/O cost is approximately:

$M + \lceil M / (B - 2) \rceil \times N$

Using more buffer pages reduces the number of times the inner relation must be scanned.

Therefore, Block Nested Loop Join is generally more efficient than the simple tuple-based Nested Loop Join.

4. Deadlock Detection Using Wait-For Graph

Pseudocode

ALGORITHM DetectDeadlock(Transactions, WaitForGraph)

BEGIN

Visited $\leftarrow$ Empty Set

RecursionStack $\leftarrow$ Empty Set

FOR EACH transaction T IN Transactions DO

IF T NOT IN Visited THEN

IF DetectCycle(T, Visited, RecursionStack, WaitForGraph)
= TRUE THEN

DISPLAY "Deadlock Detected"
RETURN TRUE

END IF

END IF

END FOR

DISPLAY "No Deadlock Detected"
RETURN FALSE

END

Cycle Detection Function

FUNCTION DetectCycle(T, Visited, RecursionStack, WaitForGraph)

BEGIN

Add T to Visited
Add T to RecursionStack

FOR EACH transaction U that T is waiting for DO

IF U NOT IN Visited THEN

IF DetectCycle(U, Visited, RecursionStack, WaitForGraph)
= TRUE THEN

RETURN TRUE

END IF

ELSE IF U IN RecursionStack THEN

RETURN TRUE

END IF

END FOR

Remove T from RecursionStack

RETURN FALSE

END

Explanation

A **Wait-For Graph (WFG)** represents transactions as nodes.

- An edge **T1** → **T2** means T1 is waiting for a resource held by T2.
- A deadlock exists when the graph contains a **cycle**.

For example:

T1 → **T2** → **T3** → **T1**

This cycle indicates a deadlock. The algorithm uses Depth First Search (DFS) to detect such cycles.

5. Basic Two-Phase Locking Protocol

Pseudocode

ALGORITHM BasicTwoPhaseLocking(Transaction T)

BEGIN

 // Growing Phase

 WHILE T requires a data item X DO

 RequestLock(T, X)

 IF Lock on X is compatible THEN

 GrantLock(T, X)

 ELSE

 Put T into WAIT state

 WAIT until lock on X is granted

 END IF

 END WHILE

```
// Shrinking Phase
AFTER T completes all lock requests DO

    WHILE T holds one or more locks DO

        ReleaseLock(T, X)

    END WHILE

END AFTER

Commit T

END
```

Lock Request Procedure

```
PROCEDURE RequestLock(T, X)

BEGIN

    IF X is not locked OR requested lock is compatible THEN

        GrantLock(T, X)

    ELSE

        Add T to WaitingQueue(X)
        Set State(T) ← WAITING

    END IF

END
```

Lock Release Procedure

```
PROCEDURE ReleaseLock(T, X)

BEGIN

    Remove lock of T on X

    IF WaitingQueue(X) is not empty THEN
```

Grant lock to next compatible transaction

END IF

END

Explanation

Basic Two-Phase Locking has two phases:

1. **Growing Phase:** The transaction can acquire locks but cannot release any lock.
2. **Shrinking Phase:** The transaction can release locks but cannot acquire new locks.

This protocol ensures conflict serializability but may lead to deadlocks.

6. Timestamp-Based Concurrency Control Using Thomas' Write Rule

Pseudocode

ALGORITHM TimestampConcurrencyControl

BEGIN

FOR EACH transaction T DO

TS(T) $\leftarrow$ GenerateUniqueTimestamp()

END FOR

END

Read Operation

PROCEDURE Read(T, X)

BEGIN

IF TS(T) < WriteTS(X) THEN

Abort(T)

Restart(T)

ELSE

Read value of X

ReadTS(X) $\leftarrow$ MAX(ReadTS(X), TS(T))

END IF

END

Write Operation Using Thomas' Write Rule

PROCEDURE Write(T, X, NewValue)

BEGIN

IF $TS(T) < ReadTS(X)$ THEN

 Abort(T)

 Restart(T)

ELSE IF $TS(T) < WriteTS(X)$ THEN

 // Thomas' Write Rule

 // Ignore this obsolete write

 Ignore Write Operation

ELSE

 Write NewValue into X

$WriteTS(X) \leftarrow TS(T)$

END IF

END

Explanation

Each data item X maintains:

- **ReadTS(X):** Largest timestamp of any transaction that successfully read X.
- **WriteTS(X):** Largest timestamp of any transaction that successfully wrote X.

Thomas' Write Rule improves basic timestamp ordering by ignoring an obsolete write instead of aborting the transaction when:

$TS(T) < WriteTS(X)$

Thus, unnecessary transaction rollbacks can be avoided.

7. ARIES Recovery Algorithm

Pseudocode

ALGORITHM ARIESRecovery(Log, Checkpoint)

BEGIN

```

// Phase 1: Analysis
TransactionTable ← Empty
DirtyPageTable ← Empty

Start from last checkpoint

FOR EACH log record FROM checkpoint to end of log DO

    Update TransactionTable

    IF page is modified THEN
        Add page to DirtyPageTable
    END IF

    IF transaction commits THEN
        Mark transaction as COMMITTED
    END IF

    IF transaction aborts THEN
        Mark transaction as ABORTED
    END IF

END FOR


// Phase 2: Redo
RedoStart ← Minimum RecLSN in DirtyPageTable

FOR EACH log record FROM RedoStart to end of log DO

    IF log record modifies page P THEN

        IF P is in DirtyPageTable AND
            $\text{LogLSN} \geq \text{RecLSN}(P)$  AND
            $\text{PageLSN}(P) < \text{LogLSN}$  THEN

            REDO the operation
            Update PageLSN(P)

        END IF

    END IF

```

END FOR

// Phase 3: Undo

UndoList $\leftarrow$ All uncommitted transactions

WHILE UndoList is not empty DO

 Select log record with largest LSN

 IF record can be undone THEN

 UNDO the operation

 Write Compensation Log Record (CLR)

 END IF

 Move to PreviousLSN

 IF transaction is completely undone THEN

 Write END record

 Remove transaction from UndoList

 END IF

END WHILE

DISPLAY "Database Recovery Completed"

END

Explanation

ARIES recovery consists of three phases:

1. Analysis Phase

Identifies:

- Active transactions
- Committed transactions
- Dirty pages
- Recovery starting points

2. Redo Phase

Repeats logged operations to ensure that all necessary changes are present in the database.

3. Undo Phase

Reverses changes made by transactions that were incomplete at the time of the crash.

Compensation Log Records (CLRs) are written during undo operations to ensure recovery remains safe even if another crash occurs.

8. Shadow Paging Recovery Algorithm

Pseudocode

ALGORITHM ShadowPagingRecovery

BEGIN

 // Initial setup

 ShadowPageTable $\leftarrow$ Copy(CurrentPageTable)

 WHILE transaction is active DO

 IF a page P is modified THEN

 NewPage $\leftarrow$ CreateCopy(P)

 Apply modifications to NewPage

 Update CurrentPageTable to point to NewPage

 END IF

 END WHILE

 IF Transaction COMMIT occurs THEN

 Save CurrentPageTable permanently

 ShadowPageTable $\leftarrow$ Copy(CurrentPageTable)

 DISPLAY "Transaction Committed Successfully"

 ELSE IF System Failure occurs THEN

 CurrentPageTable $\leftarrow$ ShadowPageTable

Discard all modified pages

DISPLAY "Database Rolled Back Successfully"

END IF

END

Explanation

Shadow Paging maintains two page tables:

1. **Shadow Page Table:** Stores the stable and consistent version of the database.
2. **Current Page Table:** Contains updates made during the current transaction.

During modification, the original page is not overwritten. A new page is created and the current page table is updated.

- **On Commit:** The current page table becomes the new stable version.
- **On Failure:** The system restores the shadow page table and discards modified pages.

This method provides simple rollback without requiring UNDO or REDO logging.

9. Immediate Update Recovery Using UNDO/REDO

Pseudocode

ALGORITHM ImmediateUpdateRecovery(Log)

BEGIN

RedoList ← Empty Set

UndoList ← Empty Set

// Step 1: Identify committed and uncommitted transactions

FOR EACH log record IN Log DO

IF log record indicates BEGIN(T) THEN

 Add T to UndoList

END IF

IF log record indicates COMMIT(T) THEN

 Remove T from UndoList

 Add T to RedoList

END IF

```

    IF log record indicates ABORT(T) THEN
        Remove T from UndoList
    END IF

END FOR

// Step 2: REDO committed transactions
FOR EACH log record FROM beginning to end DO

    IF Transaction(log record) belongs to RedoList AND
        log record contains UPDATE THEN

        Apply NewValue to database

    END IF

END FOR

// Step 3: UNDO uncommitted transactions
FOR EACH log record FROM end to beginning DO

    IF Transaction(log record) belongs to UndoList AND
        log record contains UPDATE THEN

        Restore OldValue in database

    END IF

END FOR

DISPLAY "Immediate Update Recovery Completed"

END

```

Explanation

In the Immediate Update technique, database pages may be updated before a transaction commits. Therefore, after a system crash:

- **REDO** is performed for committed transactions to ensure their updates are permanently applied.
- **UNDO** is performed for uncommitted transactions to remove their partial updates.

The log generally stores both:

<Transaction ID, Data Item, Old Value, New Value>

The old value is used for UNDO, and the new value is used for REDO.

10. Database Connectivity Manager with Connection Pooling, Query Execution, and Exception Rollback

Pseudocode

ALGORITHM DatabaseConnectivityManager

BEGIN

 ConnectionPool ← InitializeConnectionPool(MAX_CONNECTIONS)

 WHILE application is running DO

 Request ← ReceiveDatabaseRequest()

 Connection ← GetConnection(ConnectionPool)

 TRY

 BeginTransaction(Connection)

 Result ← ExecuteQuery(Connection, Request.SQL_Query)

 CommitTransaction(Connection)

 SendResultToUser(Result)

 CATCH DatabaseException

 RollbackTransaction(Connection)

 SendErrorToUser(DatabaseException)

 FINALLY

 ReturnConnectionToPool(
 ConnectionPool,
 Connection
)

END TRY

END WHILE

END

Connection Pool Functions

Get Connection

FUNCTION GetConnection(ConnectionPool)

BEGIN

IF an available connection exists THEN

 Connection ← Remove available connection from pool

ELSE IF CurrentConnections < MAX_CONNECTIONS THEN

 Connection ← CreateNewConnection()

ELSE

 WAIT until a connection becomes available

 Connection ← Remove available connection from pool

END IF

RETURN Connection

END

Return Connection

PROCEDURE ReturnConnectionToPool(ConnectionPool, Connection)

BEGIN

IF Connection is valid THEN

 Add Connection back to ConnectionPool

ELSE

 Close Connection

Create replacement connection if required

END IF

END

Query Execution

FUNCTION ExecuteQuery(Connection, SQL_Query)

BEGIN

PreparedStatement ← Prepare(SQL_Query)

Result ← Execute(PreparedStatement)

RETURN Result

END

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient